List and Tuple in Python

www.infomax.org.in

List

- A list is a collection which is ordered and changeable. In Python lists are written with square brackets.List are mutable i.e. value of lit can be modified.
- Syntax:

  <list_name>=[<value>,<value>,<value>……]
- Example:

```
list1=[]                              #empty list
List2=[4,5,6,7]                       #list of number
List3=[“ram”,”shyam”,”mohan”]         #list of string
List4=[34,56.7,”infomax”]             #mixed list
List5=[[3,4,6],[4,5,7],2]             #nested list
```

The list() Constructor

- It is also possible to use the list() constructor to make a list.

```
>>> mylist=list("infomax")
>>> print(mylist)
['i', 'n', 'f', 'o', 'm', 'a', 'x']
>>>
```

- To create a Empty list we can also use list() function.

Ex:

```
mylist=list()
```


3

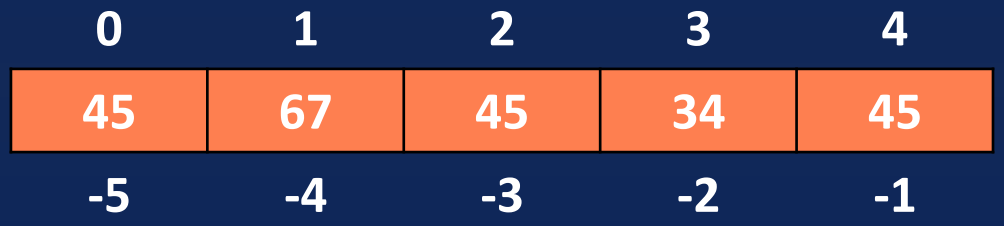
Accessing List

- All list element (items) are accessed by it index number. The list index also start with 0 so the last index is size-1. list also support the reverse indexing which start with length of list and goes to the -1.

- Syntax:

  List_name[index]

- Example:
  - Marks=[45,67,45,34,45]
  - Print(marks[1]) # 67



Traversing a String

- Traversing a list means accessing all the elements of the list one after the another by using the subscript or index.

```
mylist=[6,7,8,9]
for a in mylist:
    print(a)
```

```
mylist=[6,7,8,9]
for a in range(0,len(mylist)):
    print(mylist[a])
```

List Operations

Joining List

- Joining list means concatenate (add) two lists together. The (+) operator, applied to two lists, produce a new list that is a combination of them:

```
>>> list1=[3,4,5]
>>> list2=[7,8,9]
>>> list3=list1+list2
>>> print(list3)
[3, 4, 5, 7, 8, 9]
```

Repeating List

- Multiplying a list by an integer ‘n’ creates a new list that repeats the original list ‘n’ times. The ‘*’ operator repeats the original list the given number of times.

```
>>> list1=[2,3,5]
>>> print(list1*3)
[2, 3, 5, 2, 3, 5, 2, 3, 5]
```

Membership Operator

- There are two membership operators for List (in fact for all sequence type).
- These are **in** and **not in**
- in : Return True if a character or a substring exist in the given string; False otherwise
- **not in** : Return True if a character or a substring not exist in the given string; False otherwise

```
>>> data=[4,5,6,7,8]
>>> 5 in data
True
>>> 3in data
False
>>> 5 not in data
False
>>> 3 not in data
True
```

Comparison Operator

- Python's standard comparison operators i.e. all relational operators (<,<=,>,>=,==,!=) apply to list also.

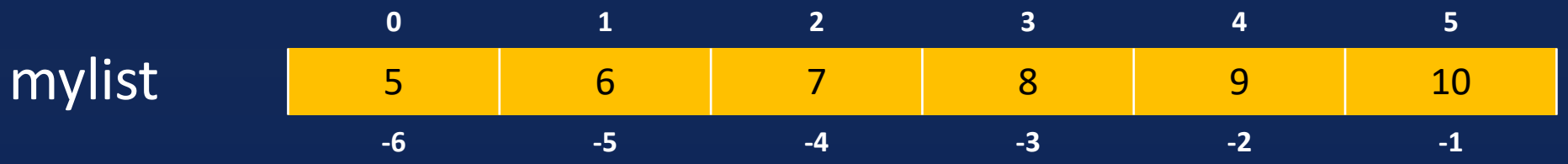
```
>>> a=[4,5,6]
>>> b=[5,3,2]
>>> c=[4,2,8]
```

```
>>> a>b
False
>>> a>c
True
>>> a<b
True
>>> a<c
False
>>> b<c
False
```

List Slices

- Slicing refers to a part of the list, where list are sliced using a range of indices (index).
- Syntax:

  list_name[start:end:step] (default step is 1).



mylist[0:3]	5 6 7	mylist[2:5]	7 8 9
mylist[-7:-3]	5 6 7 8	mylist[:5]	5 6 7 8 9
mylist[3:]	8 9 10	mylist[1:5:2]	6 8
mylist[]	5 6 7 8 9 10	mylist[::-1]	10 9 8 7 6 5

Aliasing

- A circumstance where two or more variables refers to the same object is called aliasing.
- If a refers to an object and you assign a to b, then both variables refer to the same object.

```
>>> d1=[5,6,7]
>>> d2=d1
>>> d2[0]=30
>>> d1
[30, 6, 7]
>>> d2
[30, 6, 7]
```

Len( )

- it returns the length of list i.e. number of items in list.

```
>>> data=[4,5,6]
>>> print(len(data))
3
```


13

count()

- Count(item): to find out the number of times a value occurs in a list.

```
>>> mylist=[3,3,4,3,4,5]
>>> mylist.count(3)
3
```

```
>>> mylist=[7,7.0,"7",6,8]
>>> mylist.count(7)
2
```

Index()

- It return the index first occurrence of searched item in list.
- If start and value is not specified the it search the item in complete list and if it specified the it search a item in a sub list.
- Syntax: list.index(item)
  list.index(item,start position)
  list.index(item,start index,end index)

```
>>> mylist=[11,22,33,44,55,22,11]
>>> mylist.index(22)
1
>>> mylist.index(22,4)
5
```

append( )

- Append() used add an item to the end of a list.
- This method change the list in place.
- Synyax: list.append(item)

```
>>> data=[2,3,4]
>>> data.append(1)
>>> data
[2, 3, 4, 1]
```

extend()

- Extend() function is used to add the items of one list i.e. iterable object to an existing list.
- In other words, all the item of a list are added at the end of already created list.
- Syntax: list.extend(list2)

```
>>> list1=[1,2,3]
>>> list2=[4,5,6]
>>> list1.extend(list2)
>>> list1
[1, 2, 3, 4, 5, 6]
```

Copy()

- It create a duplicate list with same elements with specified list.
- Note when we use = operator to copy the list it only create a alias of list i.e. no duplicate list will be created both variable access the same list.
- Syntax: list2=list1.copy()

```
>>> list1=[4,5,6]
>>> list2=list1.copy()
>>> list2.append(4)
>>> list1
[4, 5, 6]
>>> list2
[4, 5, 6, 4]
```

Insert()

- It insert an item at a specified position and move the remaining items to the right.
- Syntax: *list.insert(position,element)*
- Note: position mentioned should be within the range of list.

```
>>> mylist=[5,6,7,9]
>>> mylist.insert(2,8)
>>>
>>> mylist
[5, 6, 8, 7, 9]
>>> mylist.insert(20,8)
>>> mylist
[5, 6, 8, 7, 9, 8]
```

Pop()

- The pop() removes the item at the given position in the list, and returns it.
- If no index is specified , pop() removes and returns “rightmost” i.e. the last item in the list.
- Syntax: list.pop(<index>)

```
>>> mylist=[5,6,7,8]
>>> mylist.pop(2)
7
>>> mylist
[5, 6, 8]
>>> mylist.pop()
8
>>> mylist
[5, 6]
```

Remove()

- The remove() method is used to remove the first occurrence of a value.
- If the specified element is not found in list the it will generate a error.
- Syntax: list.remove(item)

```
>>> mylist=[5,6,7,8]
>>> mylist.remove(6)
>>> mylist
[5, 7, 8]
>>> mylist.remove(2)
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    mylist.remove(2)
ValueError: list.remove(x): x not in list
```

Reverse()

- The reverse() function reveres the order of items/elements in the list.
- Syntax: list.reverse()

```
>>> mylist=[5,6,7,8]
>>> mylist.reverse()
>>> mylist
[8, 7, 6, 5]
```

Sort()

- Sort method is used to arrange the items of list ascending or descending order.
- Syntax: list.sort()
  list.sort(revrese=True)

```
>>> mylist=[5,4,6,7,8]
>>> mylist.sort()
>>> mylist
[4, 5, 6, 7, 8]
>>> mylist.sort(reverse=True)
>>> mylist
[8, 7, 6, 5, 4]
```

Clear()

- Clear() function is used to remove all the items from list.
- It is equivalent to del list_name[:]
- Syntax: list.clear()

```
>>> mylist=[4,5,6,7]
>>> mylist.clear()
>>> mylist
[]
```

Min() and max()

- Min() returns the smallest value from the list.
  - Syntax: min(list)
- Max() return the largest number form the list.
  - Syntax: max(list)

```
>>> mylist=[4,5,6,7]
>>> min(mylist)
4
>>> max(mylist)
7
```

Sum()

- It return the sum off all elements present in list.
- Syntax: sum(list)

```
>>> mylist=[6,2,3,5]
>>> sum(mylist)
16
```

Del Statement:

- Dele statement allows you to remove specific items from the list or to remove all items identified by a slice.
- Syntax: del list[i]
  del[i:j]

```
>>> mylist=[5,6,6,7,8,4]
>>> del mylist[2:4]
>>> mylist
[5, 6, 8, 4]
>>> del mylist[2]
>>> mylist
[5, 6, 4]
```

Tuple in Python

Tuple

- A tuple is a sequence of values, which can be of any type and they are indexed by integer.
- Tuples are just like list, but we can't change values of tuples in place. Thus tuples are **immutable**. The index value of tuple starts from 0.
- A tuple consists of a number of values separated by commas.
- Example:
  - T1=1,2,3,4
  - T2=(1,2,2,3)
  - T3=('ram','shyam',1,3,4)

29

Tuple Creation form existing sequence

- tuple() constructor/function is used to create tuple from existing sequence i.e. from list or string.
- Example :
  - t1=tuple("infomax") # ('i','n','f','o','m','a','x')
  - t2=tuple([5,6,7,8]) # (5,6,7,8)

30

Tuple Operations

Joining Tuple

- We can add single element or join existing tuple using + operator.

```
>>> t1=(5,7,8,9)
>>> t1=t1+(2,)
>>> t1
(5, 7, 8, 9, 2)
```

```
>>> t1=(1,2,3)
>>> t2=(4,5,6)
>>> t3=t1+t2
>>> print(t3)
(1, 2, 3, 4, 5, 6)
```

Replication

- * operator is used to replicate the tuple elements.

```
>>> a=(3,4,5)
>>> a*3
 (3, 4, 5, 3, 4, 5, 3, 4, 5)
```

Membership

- in and not in operator is also used in tuple to check the existence of particular element.

```
>>> a=(5,6,8,9,2,4)
>>> 8 in a
True
>>> 7 in a
False
>>> 8 not in a
False
>>> 7 not in a
True
```

Comparison

- All relation operator are also used in tuple to compare two tuples.

```
>>> a=(3,4,5)
>>> b=(3,5,4)
>>> a==b
    False
>>> a>b
    False
>>> a<b
    True
>>> a>=b
    False
>>> a<=b
    True
>>> a!=b
    True
```

Tuple Slices

- Slice operator works on Tuple also. This is used to display more than one selected value on the output screen. Slices are treated as boundaries and the result will contain all the elements between boundaries.

Seq = T [start: stop: step]

```
>>> T=(9,8,7,6,5,4,3,2,1)
>>> T[:5]
(9, 8, 7, 6, 5)
>>> T[5:]
(4, 3, 2, 1)
>>> T[3:6]
(6, 5, 4)
>>> T[::-1]
(1, 2, 3, 4, 5, 6, 7, 8, 9)
>>> T[::2]
(9, 7, 5, 3, 1)
```

Tuple Functions

- len( ) : It returns the number of items in a tuple.
- max( ) : It returns its largest item in the tuple.
- min( ) : It returns its smallest item in the tuple.
- sum() : to fins the sum of all elements in tuple
- count() : to count the number of occurrences of particular element.
- index() : to find the index (position) of particular element in tuple.
- sorted() : it return the sorted version of tuple in list format.

Packing and unpacking tuple

- You can also create a tuple without parentheses. This is called tuple packing.
- Multiple assignment can be performed using tuples that known as unpacking.

Set in Python

Set in Python

- a **set** is an **unordered**, **mutable** collection of **unique** elements.

- Ex:
- Set1={4,6,7,8}

Creating a Set

- Using curly braces

```
s = {1, 2, 3}
```

- Using the set() constructor

```
s2 = set([1, 2, 2, 3, 4])
print(s2)  # Output: {1, 2, 3, 4}
```

Set Operations

- Union
  - a = {1, 2, 3}
  - b = {3, 4, 5}
  - print(a | b) # {1, 2, 3, 4, 5}
  - print(a.union(b)) # Same
- Intersection
  - print(a & b) # {3}
  - print(a.intersection(b))
- Difference

  print(a - b) # {1, 2}

  print(b.difference(a)) # {4, 5}

Common Set Methods

Method	Description
add(elem)	Adds an element
remove(elem)	Removes element; error if not present
discard(elem)	Removes element; no error if missing
pop()	Removes & returns an arbitrary element
clear()	Empties the set
update(iterable)	Adds multiple elements
copy()	Returns a shallow copy

Thank You

Address: G. R. Complex Preetam Nagar, Dhoomanganj
Prayagraj Uttar Pradesh 211011

Mobile No.: 9598948810,8874588766,9532878816
Website : www.infomax.org.in